

Kipu

Un lenguaje de dominio específico para reglas tributarias y montos monetarios



Analizadores léxico, sintáctico y semántico (parcial) en ANTLR4

Integrantes: Fabrizio Bussalleu Salcedo · Jose Luis Sancho Navarro

Docente: Jose Luis Soncco Alvarez · UPC, Lima, octubre de 2026

Las reglas tributarias viven en Excel o escondidas en código de propósito general

IGV 18 %, detracciones del SPOT (12 % sobre operaciones mayores a S/ 700), retenciones, percepciones y tipo de cambio: todo se programa a mano en cada sistema de facturación.

01

Punto flotante

$0.1 + 0.2 \neq 0.3$. El redondeo a céntimos queda disperso por todo el código.

02

Monedas mezcladas

Nada impide sumar soles con dólares; el error aparece en la conciliación o en una fiscalización.

03

Reglas ilegibles

Un contador no puede auditar la regla, aunque SUNAT la cambie con cada resolución.

04

Errores tardíos

Los fallos se detectan en ejecución (o nunca), no cuando se escribe la regla.

Las bibliotecas de dinero (JSR 354, py-moneyed) validan la moneda **en ejecución**. No identificamos un lenguaje en español donde el monto con moneda sea un **tipo verificado en compilación**.

Kipu: montos, monedas y porcentajes como tipos nativos

```
const IGV: porcentaje = 18%;
const UMBRAL: monto<PEN> = 700.00 PEN;

regla calcularIgv(base: monto<PEN>): monto<PEN> {
    retornar base * IGV;
}

var subtotal: monto<PEN> = 250.00 PEN * 3;
var igv: monto<PEN> = calcularIgv(subtotal);
var total: monto<PEN> = subtotal + igv;
si (total > UMBRAL) {
    mostrar("Detracción:", total * 12%);
}
var usd: monto<USD> = convertir(total, USD, 3.75);
```

Sintaxis en español

Legible para quien define la regla: si, mientras, regla, retornar.

Tipos de dominio

monto<PEN> y monto<USD> son tipos distintos; solo convertir cambia la moneda.

Errores en compilación

Sumar PEN con USD o asignar texto a un monto se rechaza antes de ejecutar.

Un compilador que verifica la coherencia contable antes de ejecutar

General: diseñar e implementar un compilador para Kipu con ANTLR4 y LLVM que verifique en compilación la coherencia de tipos y monedas.

OE1 · HITO 1

Léxico y construcciones

Tipos básicos más monto<M> y porcentaje.

OE2 · HITO 1

Gramática

BNF por construcción validada con derivaciones más a la izquierda.

OE3 · HITO 1

Lexer y parser

ANTLR4 con errores en español, línea y columna.

OE4 · HITOS 1-2

Semántica

Tabla de símbolos por ámbitos; 5 errores hoy, 6 o más en el Hito 2.

OE5 · HITO 3

LLVM IR

Montos como enteros de 64 bits en céntimos: cero error de flotante.

Siete construcciones, cinco ejemplos cada una (35 validados con el parser)

Construcción	Ejemplo representativo
G1 Declaración	<code>var subtotal: monto<PEN> = 1250.50 PEN;</code>
G2 Asignación (=, +=, -=)	<code>saldo -= 150.00 PEN;</code>
G3 Expresiones	<code>total >= 700.00 PEN y no exonerado</code>
G4 Selectivas	<code>si (esExportacion) { igv = 0.00 PEN; } sino { igv = base * 18%; }</code>
G5 Iterativas	<code>para mes desde 1 hasta 12 { total += cuota; }</code>
G6 Reglas, retorno y llamadas	<code>regla esMayor(a: entero, b: entero): booleano { retornar a > b; }</code>
G7 Entrada y salida	<code>mostrar("IGV", calcularIgv(base), "Neto", neto);</code>

Precedencia (mayor a menor): (), llamadas y convertir · menos unario · * / mod · + - · relacionales · igualdad · no · y · o

52 tokens en Kipu.g4, con prioridad de palabras clave y coincidencia más larga

Categoría	Token: lexemas
Palabras clave (26)	VAR: var · SI: si · REGLA: regla ...
Monedas	MONEDA: PEN, USD, EUR
Operadores (13)	MAS_ASIG: += · MENOR_IGUAL: <= ...
Delimitadores (7)	PAR_IZQ: (· PUNTO_COMA: ; ...
Literales (4)	LIT_PORCENTAJE: 18% · NUM_DECIMAL: 3.75
Identificadores	ID: subtotal, IGV, año

Símbolos fuera del alfabeto (#, @, \$) y cadenas sin cerrar producen el error **LEX-01** con línea y columna.

```
$ python src/kipu.py ej.kp --tokens
```

```
#  Lín:Col  Token      Lexema
1  1:1      VAR        var
2  1:5      ID         subtotal
3  1:13     DOS_PUNTOS :
4  1:15     MONTO      monto
5  1:20     MENOR      <
6  1:21     MONEDA     PEN
7  1:24     MAYOR      >
8  1:26     ASIG       =
9  1:28     NUM_DECIMAL 1250.50
10 1:36     MONEDA     PEN
11 1:39     PUNTO_COMA ;
```

Gramática libre de contexto: un no terminal por nivel de precedencia

```

expr → expr o conj | conj
conj → conj y neg | neg
neg → no neg | igu
igu → igu op_igu rel | rel
rel → rel op_rel arit | arit
arit → arit op_sum term | term
term → term op_mul unario | unario
unario → - unario | prim
prim → ( expr ) | convertir ( expr , moneda , expr ) |
llamada | lit | id

```

Derivación más a la izquierda de `cantidad = 5;`

```

asig
⇒ id op_asig expr ;
⇒ id = expr ;
⇒ id = conj ;
⇒ id = neg ;
⇒ id = igu ;
⇒ id = rel ;
⇒ id = arit ;
⇒ id = term ;
⇒ id = unario ;
⇒ id = prim ;
⇒ id = lit ;
⇒ id = num_ent ;

```

En Kipu.g4 la misma precedencia se expresa con recursión por la izquierda, que ANTLR4 reescribe internamente. Los errores sintácticos salen como **SIN-01**, en español.

28 derivaciones generadas y verificadas automáticamente, no a mano



35 / 35

ejemplos aceptados por la BNF y por el parser ANTLR

28

derivaciones más a la izquierda (4 por gramática) en el Anexo A

0

ambigüedades; los programas completos también cumplen la BNF

Visitor con tabla de símbolos por ámbitos y sistema de tipos monetario

Código	Error verificado	Ejemplo
SEM-01	Identificador no declarado	<code>total = subtotal + igv;</code>
SEM-02	Redeclaración en el mismo ámbito	<code>var total ... (dos veces)</code>
SEM-03	Monedas incompatibles	<code>soles + dolares</code>
SEM-04	Tipos incompatibles	<code>precio * precio</code>
SEM-05	Modificar una constante	<code>IGV = 10%;</code>

Reglas de tipos

`monto<M> ± monto<M> → monto<M>`

`monto × entero | decimal |
porcentaje → monto`

`monto<M> / monto<M> → decimal`

`entero → decimal (ensanchamiento)`

`convertir(monto<A>, B, tc) →
monto`

Un error marca la subexpresión como desconocida para evitar errores en cascada.

Cuatro errores distintos, reportados en una sola pasada con línea y columna

```
// Consolidado anual de ventas
const UIT: monto<PEN> = 5350.00 PEN;
var ventas: monto<PEN> = 0.00 PEN;
var ventasUsd: monto<USD> = 0.00 USD;
para mes desde 1 hasta 12 {
    UIT = 5500.00 PEN;
    ventas += ventasUsd;
    mostrar(totalAnual);
    var mes: entero = 1;
}
```

```
$ python src/kipu.py errores.kp
[1/3] Análisis léxico      OK
[2/3] Análisis sintáctico  OK
[3/3] Análisis semántico   4 error(es)

[SEM-05] línea 6:5  No se puede modificar
           la constante 'UIT'.
[SEM-03] línea 7:15 Monedas incompatibles:
           monto<PEN> con monto<USD>
[SEM-01] línea 8:13 'totalAnual' no
           ha sido declarado.
[SEM-02] línea 9:9  'mes' ya fue declarado
           en este ámbito (línea 5).
```

68 casos de prueba automáticos, todos aprobados

Analizador	Casos	Resultado
Léxico	6 archivos + 5 secuencias de tokens	11 / 11
Sintáctico	35 ejemplos + 3 programas válidos + 9 inválidos	47 / 47
Semántico	4 válidos + 6 inválidos (19 errores esperados)	10 / 10

Comando: `python -m unittest discover -s tests -v`

Repositorio: [https://github.com/\[usuario\]/kipu](https://github.com/[usuario]/kipu)

```
kipu/
├── grammar/Kipu.g4
├── src/kipu.py
├── src/semantico.py
├── src/generated/
├── tests/
│   ├── lexico/
│   ├── sintactico/
│   └── semantico/
├── scripts/derivaciones.py
├── ejemplos/facturacion.kp
└── docs/
```

Del front end al código ejecutable

HITO 1 · ACTUAL

Front end parcial

Léxico, sintaxis, 5 errores semánticos, 68 pruebas y derivaciones verificadas.

HITO 2 · SEMANA 12

Semántica completa

Condiciones booleanas, tipo de retorno, aridad y tipos de argumentos, porcentaje fuera de rango, división entre cero. Arquitectura y plan de validación.

HITO 3 · SEMANA 15

Generación con LLVM

LLVM IR con llvmlite: montos como i64 en céntimos, redondeo explícito y reglas como funciones nativas.

Kipu: si la regla no tiene sentido contable, no compila.